

worry (yet) about the representation of `Parser`. As you go, you'll undoubtedly discover additional combinators and idioms, notice and factor out common patterns, and so on. Use the skills you've been developing throughout this book, and have fun! If you get stuck, you can always consult the answers.

Here's some minimal guidance:

- Any general-purpose combinators you discover can be added to the `Parsers` trait directly.
- You'll probably want to introduce combinators that make it easier to parse the tokens of the JSON format (like string literals and numbers). For this you could use the `regex` primitive we introduced earlier. You could also add a few primitives like `letter`, `digit`, `whitespace`, and so on, for building up your token parsers.

Consult the hints if you'd like more guidance. A full JSON parser is given in the file `JSON.scala` in the answers.

9.5 *Error reporting*

So far we haven't discussed error reporting at all. We've focused exclusively on discovering a set of primitives that let us express parsers for different grammars. But besides just parsing a grammar, we want to be able to determine how the parser should respond when given unexpected input.

Even without knowing what an implementation of `Parsers` will look like, we can reason abstractly about what information is being specified by a set of combinators. None of the combinators we've introduced so far say anything about *what error message* should be reported in the event of failure or what other information a `ParseError` should contain. Our existing combinators only specify what the grammar is and what to do with the result if successful. If we were to declare ourselves done and move to implementation at this point, we'd have to make some arbitrary decisions about error reporting and error messages that are unlikely to be universally appropriate.



EXERCISE 9.10

Hard: If you haven't already done so, spend some time discovering a nice set of combinators for expressing what errors get reported by a `Parser`. For each combinator, try to come up with laws specifying what its behavior should be. This is a very open-ended design task. Here are some guiding questions:

- Given the parser `"abra".**(" ".many).**("cadabra")`, what sort of error would you like to report given the input `"abra cAdabra"` (note the capital 'A')? Only something like `Expected 'a'?` Or `Expected "cadabra"?` What if you wanted to choose a different error message, like `"Magic word incorrect, try again!"`?